-- method GET/POST/PUT : spécifier une méthode précise que l'on souhaite utiliser
-- data permet d'envoyer des données aux formats JSON/XML ou même par exemple des chaine de connection username&password.

Gérer les erreurs de SQLMap :

-- parse-errors : cette options permet d'afficher les erreurs renvoyés par le SGBD nous donnant ainsi une meilleur idée de la situation
-t /path/to/traffic.txt : cette option permet de stocker l'intégralité des requete (le trafic) sorties et entrées permettant une analyse très précise des potentielles erreurs
-v 6 : définir une haute verbosité à l'outil permet de suivre précisément ses requetes et donc de mieux cerner une potentielle erreur.
-- proxy="http://127.0.0.1:8080" envoyer le trafic a travers un proxy comme burpsuite peux permettre de mieux visualiser et étudier le traffic SQLMap.

Affiner ses attaques :

Dans la majorité des cas SQLMap est fais pour fonctionner avec cette syntaxe de base ... Mais certains cas nécessitent un ciblage plus avancé...
Chaque charges utiles (payloads) SQLMap n'est pas du code SQL aléatoire, elle est systématiquement décomposé en 5 composants logique précis.

[Prefix] [Union/Error/Blind/code] [Expression] [Condition] [Suffix]

Prefix : Il sert à fermer proprement la requete légitime du serveur web pour que le code SQL qui suit soit interprété par le serveur. Exemple : ' ou) ou ')
Si le site utilise une syntaxe complexe SQLMap peux échouer à fermer la requête originale on peux forcer le prefix manuellement - je dis bien le le bloc est un prefix pas un ensemble de prefix possible : exemple : --prefix="%')))"

Pareil pour les autres composant du payload ils sont modifiables mais encore une fois les payloads par défault sont très complets

Par défault SQLmap possède un ensemble predefini des paires prefix/suffix les plus courant classé en probabilité de succès sur un cible vulnérable c'est pour cela qu'on à l'option level, plus le level est elevé moins les payloads ont de probabilités de succès les option s'étendent de 1 à 5 : Ex: -- level 3
Il y a de même un classement de risk de 1 à 3 sur base du risque de causer des problèmes dans la base de données ciblés. Ex: -- risk 1
Les niveau de risk et de payload plus avancé en plus de créer une masse de trafic peuvent vraiment foutre le bordel dans une DB vulnerable, à n'utiliser qu'avec modération et en connaissance de cible ... mais ces options plus avancés peuvent s'avérer utiles dans certains cas (Ex bypass de formulaires de login).

Pour la détection des reponse bien que bien par défault on peux la personnaliser par exemple pour filtrer la détection des réponses TRUE à un code spécifique -- code=200 ou encore sur l'apparition d'une chaine de caractère précise dans la reponse avec -- string=

On peux aussi fixer les techniques utilisé à un certains nombres car celles qui nous intéressent pas ne généreraient que du trafic inutile par exemple si on ne veut utiliser que les (basé sur les boulean) les (basées sur les erreurs et sur les (requetes union) : -- technique=BEU

Comme on l'a vu sur les bases de SQL les injection UNION based nécessite un nombre exacte de colone dans la requete SQL vulnérable. si les valeurs par défaut d'SQLMap echoue on peux le forcer manuellement. Exemple : --union-cols=17